

VULNERABILITY ASSESSMENT REPORT

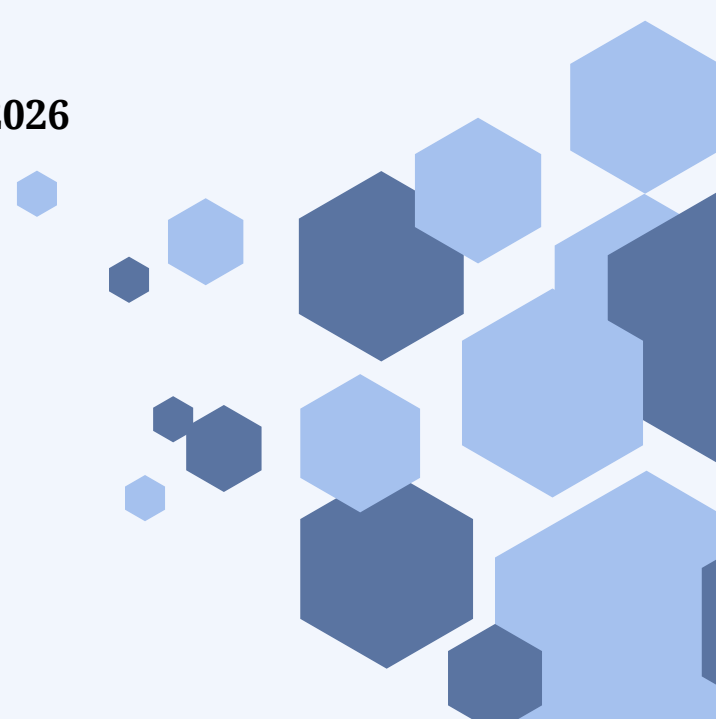
OWASP ZAP Security Assessment

**Target Website:
Smit Cyber Security Demo Website**

**Assessment Tool:
OWASP ZAP v2.17.0**

**Assessment Type:
Web Vulnerability Assessment**

**Date:
06 August 2026**



**Prepared By
Smit Borkhetaria**

Executive Summary

This report presents the results of a web vulnerability assessment performed on the Smit Cyber Security Demo Website using OWASP ZAP (Zed Attack Proxy) v2.17.0. The purpose of this assessment was to identify common web security weaknesses through passive scanning and to understand how professional vulnerability assessment tools are used during security testing.

During the assessment, the website was accessed through the OWASP ZAP proxy, allowing HTTP and HTTPS traffic to be analyzed. The scan identified several security header-related findings, including the absence of Content Security Policy (CSP), X-Frame-Options, Strict-Transport-Security (HSTS), and X-Content-Type-Options headers. Informational findings related to cache behavior were also observed.

These findings are common in demonstration websites and provide valuable insight into web application security best practices. Appropriate mitigation recommendations have been included to explain how these issues can be addressed in production environments.

This project demonstrates the practical use of OWASP ZAP for identifying web application security issues and highlights the importance of secure HTTP response headers in improving overall web security.

Introduction

Web applications are among the most common targets of cyber attacks because they are accessible over the Internet and often process sensitive user information. Vulnerabilities in web applications can lead to unauthorized access, data breaches, website defacement, and other security incidents. Therefore, regular vulnerability assessments are essential for identifying and reducing security risks.

OWASP ZAP (Zed Attack Proxy) is an open-source web application security testing tool developed by the Open Worldwide Application Security Project (OWASP). It helps security professionals, developers, and students discover common security vulnerabilities in web applications through passive and active scanning techniques.

In this internship task, a demo website named Smit Cyber Security Demo Website was created and assessed using OWASP ZAP. The objective was to perform a passive vulnerability assessment, identify security weaknesses, analyze the scan results, and recommend appropriate mitigation measures based on industry best practices.

This report documents the complete assessment process, the vulnerabilities identified, their potential impact, and the recommended security improvements.

Title

Objectives

- To understand the fundamentals of web application vulnerability assessment.
- To perform a passive security assessment using OWASP ZAP.
- To identify security weaknesses present in the demo web application.
- To analyze the findings generated by OWASP ZAP and understand their security impact.
- To study the importance of HTTP security headers in protecting web applications.
- To recommend appropriate mitigation techniques based on industry security best practices.
- To gain practical experience in documenting a professional vulnerability assessment report.

Scope Of Assessment

The scope of this assessment was limited to the Smit Cyber Security Demo Website, which was developed as part of the internship project. The assessment was performed using OWASP ZAP v2.17.0 to identify common web application security issues through passive scanning.

Assessment Included :

- Analysis of HTTP request and response traffic.
- Identification of missing or insecure HTTP security headers.
- Review of informational security findings generated by OWASP ZAP.
- Evaluation of the website's basic security posture.

Assessment Excluded :

- Exploitation of vulnerabilities.
- Authentication or login testing.
- Database security testing.
- Source code review.
- Denial-of-Service (DoS) testing.
- Any activity that could damage or disrupt the application.

Tools & Assessment Environment

The vulnerability assessment was performed in a controlled local environment using industry-standard security tools. The following tools were used during different stages of the assessment process.

Tool	Purpose	Status
OWASP ZAP v2.17.0	Passive vulnerability assessment of the web application	Used
Mozilla Firefox	Browser configured with ZAP proxy	Used
Visual Studio Code	Development of the demo website	Used
Live Server Extension	Hosted the website locally for testing	Used
Nmap	Basic network and port scanning	Used

Assessment Environment :

The assessment was conducted on a locally hosted demo website using Visual Studio Code and the Live Server extension. Mozilla Firefox was configured to route traffic through OWASP ZAP, allowing HTTP requests and responses to be captured and analyzed. Nmap was used to perform basic network reconnaissance before the web application assessment.

Assessment Methodology

Introduction

The vulnerability assessment was carried out using a structured methodology to ensure that the web application was tested in a controlled and systematic manner. The assessment process included environment preparation, website deployment, network reconnaissance, proxy configuration, passive vulnerability scanning, and analysis of the generated findings.

Assessment Process

1. Website Development

A simple demo website was created using HTML and hosted locally using Visual Studio Code and the Live Server extension.

2. Network Reconnaissance

Nmap was used to identify the available network services and verify connectivity before performing the web application assessment.

3. OWASP ZAP Configuration

OWASP ZAP was configured as a local proxy, and Mozilla Firefox was configured to route web traffic through the proxy.

4. Passive Vulnerability Assessment

The website was browsed through Firefox while OWASP ZAP captured and analyzed HTTP requests and responses.

5. Findings Analysis

The vulnerabilities reported by OWASP ZAP were reviewed, categorized according to their severity, and analyzed to understand their security impact.

6. Documentation

The complete assessment process, findings, recommendations, and supporting screenshots were documented in this report.

Target Website Overview

Introduction

A demo website named "Smit Cyber Security Demo Website" was developed for the purpose of this internship project. The website was hosted locally using Visual Studio Code and the Live Server extension. It served as the target application for the OWASP ZAP vulnerability assessment. The objective was to understand how security assessment tools identify common web application security issues in a controlled testing environment.

Website Features

- Developed using HTML.
- Hosted locally using the Live Server extension.
- Accessible through a web browser.
- Used exclusively for security testing and learning purposes.
- Served as the target application for OWASP ZAP.

Target Website Overview

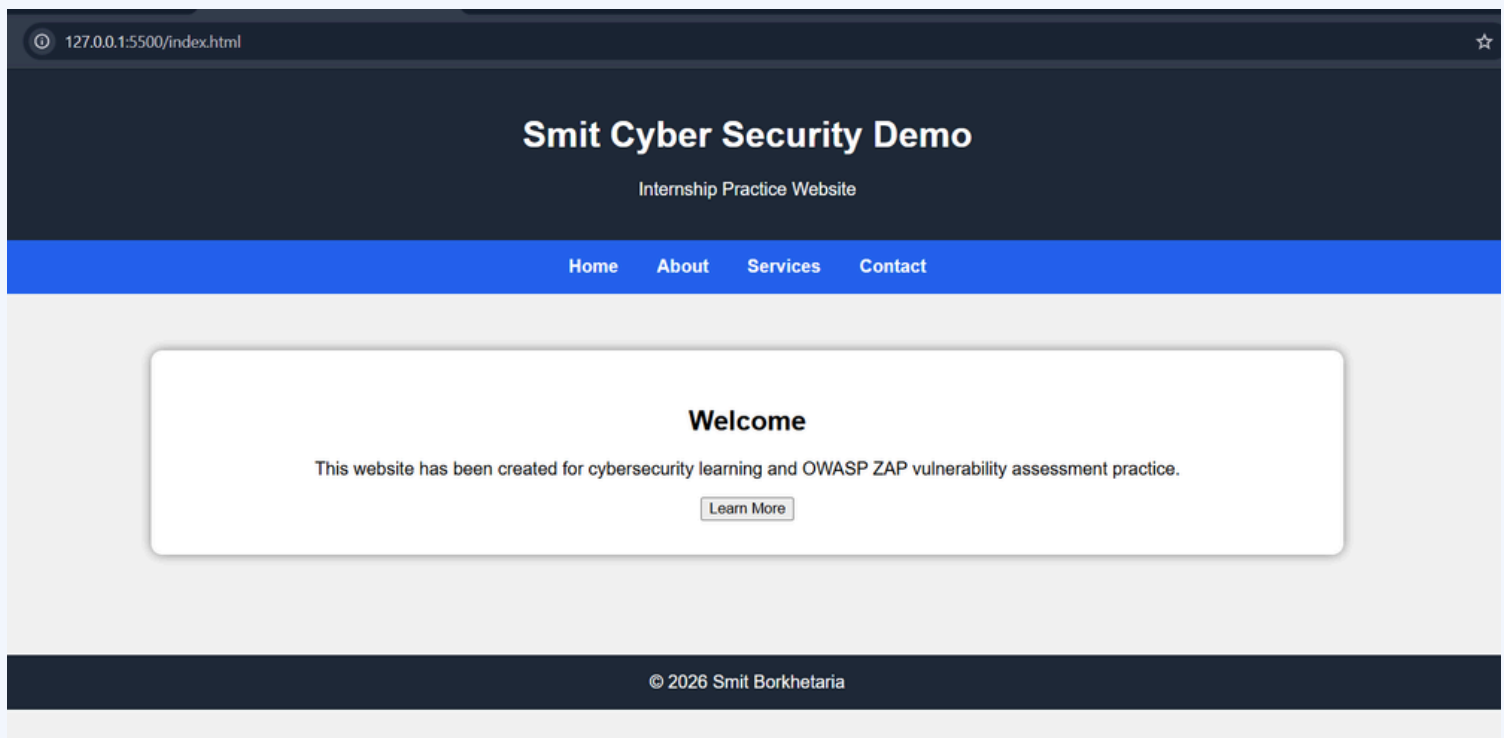


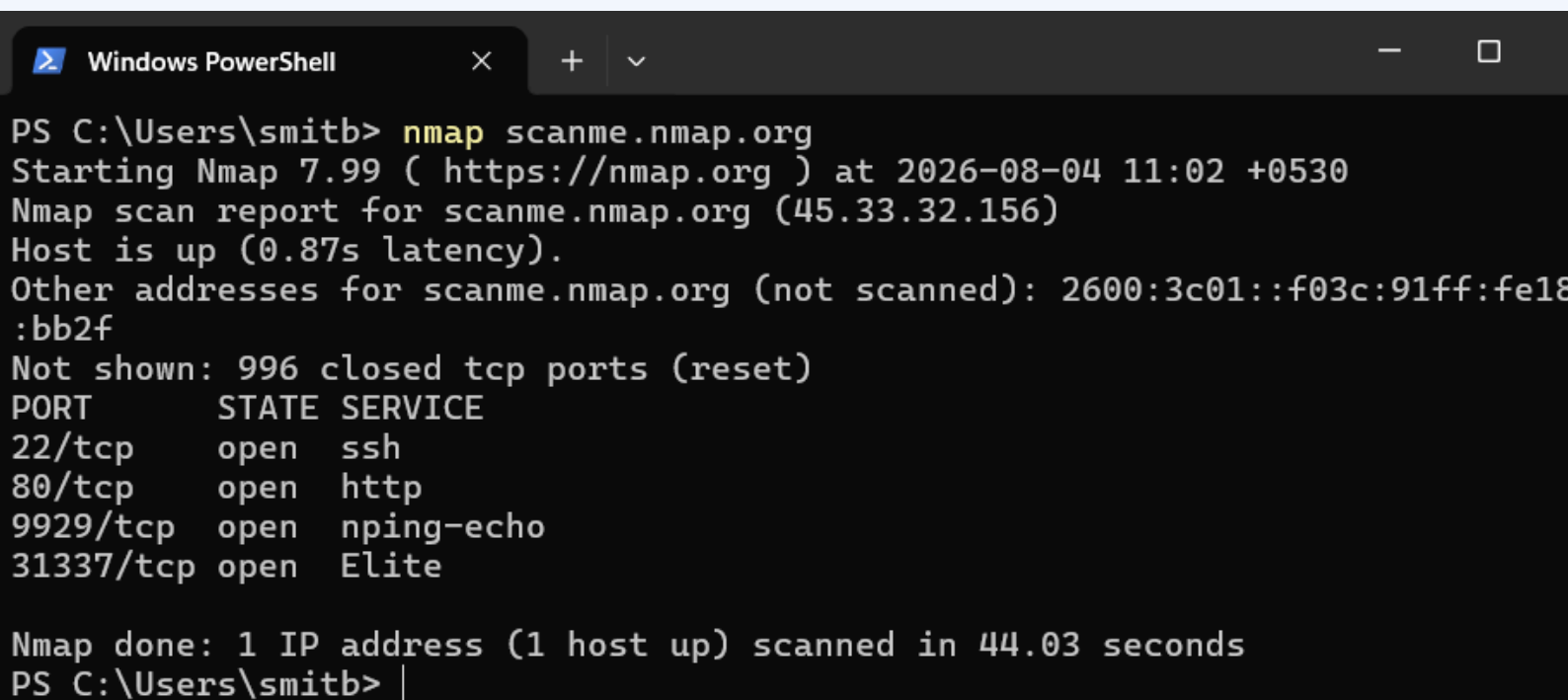
Figure 1: Homepage of the Smit Cyber Security Demo Website hosted on the local development server (127.0.0.1:5500).

Network Reconnaissance Using Nmap

Introduction

Before performing the web application security assessment, a basic network reconnaissance was carried out using Nmap (Network Mapper). The objective of this step was to verify that the target system was reachable and to identify the services available on the local host. Network reconnaissance provides an overview of the target environment and helps in understanding which services are exposed before conducting further security testing.

NMAP Scan Screenshot



```
Windows PowerShell
PS C:\Users\smittb> nmap scanme.nmap.org
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-04 11:02 +0530
Nmap scan report for scanme.nmap.org (45.33.32.156)
Host is up (0.87s latency).
Other addresses for scanme.nmap.org (not scanned): 2600:3c01::f03c:91ff:fe18:bb2f
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
9929/tcp   open  nping-echo
31337/tcp  open  Elite

Nmap done: 1 IP address (1 host up) scanned in 44.03 seconds
PS C:\Users\smittb> |
```

Figure 2: Nmap scan results showing the open services available on the local host.

Analysis

The Nmap scan confirmed that the target system was accessible and that the required service for the locally hosted website was available. This verification ensured that the assessment environment was correctly configured before the OWASP ZAP vulnerability assessment was performed.

OWASP ZAP Configuration

Introduction

Before starting the vulnerability assessment, OWASP ZAP was configured as a local proxy to intercept and analyze web traffic between the browser and the target website. Mozilla Firefox was configured to use the OWASP ZAP proxy running on localhost:8080. This allowed all HTTP requests and responses to pass through ZAP for passive security analysis.

Configuration Steps

Browser Configuration :

- Configured Mozilla Firefox to use localhost as the proxy server.
- Set the proxy port to 8080.
- Verified that all browser traffic was routed through OWASP ZAP.

OWASP ZAP Setup :

- Started OWASP ZAP.
- Confirmed the proxy listener was active on localhost:8080.
- Opened the demo website through Firefox.
- Verified that the website appeared in the OWASP ZAP Sites tree.

Configure Proxy Access to the Internet

☐ No proxy

☐ Auto-detect proxy settings for this network

☐ Use system proxy settings

☒ Manual proxy configuration

HTTP Proxy Port

☐ Also use this proxy for HTTPS

HTTPS Proxy Port

SOCKS Host Port

☐ SOCKS v4 ☒ SOCKS v5

☐ Automatic proxy configuration URL

No proxy for

Figure 3: Mozilla Firefox configured to route traffic through the OWASP ZAP proxy.

History										
Filter: OFF Export										
ID	Source	Req. Timestamp	Method	URL	Code	Reason	RTT	Size Resp. Body	Highest Alert	Note
1	Proxy	05/08/26, 10:51:52 am	GET	https://example.com/	200	OK	479 ms	559 bytes	Medium	

Figure 4: OWASP ZAP successfully capturing traffic from the locally hosted demo website.

Passive Scan Results

Introduction

After configuring OWASP ZAP and browsing the target website, a passive vulnerability scan was performed. Passive scanning analyzes HTTP requests and responses without attempting to exploit the application. The scan identified several security-related issues, mainly involving missing HTTP security headers and browser security best practices.

Summary Table

Severity	Number Of Findings
High	0
Medium	0
Low	4
Informational	2

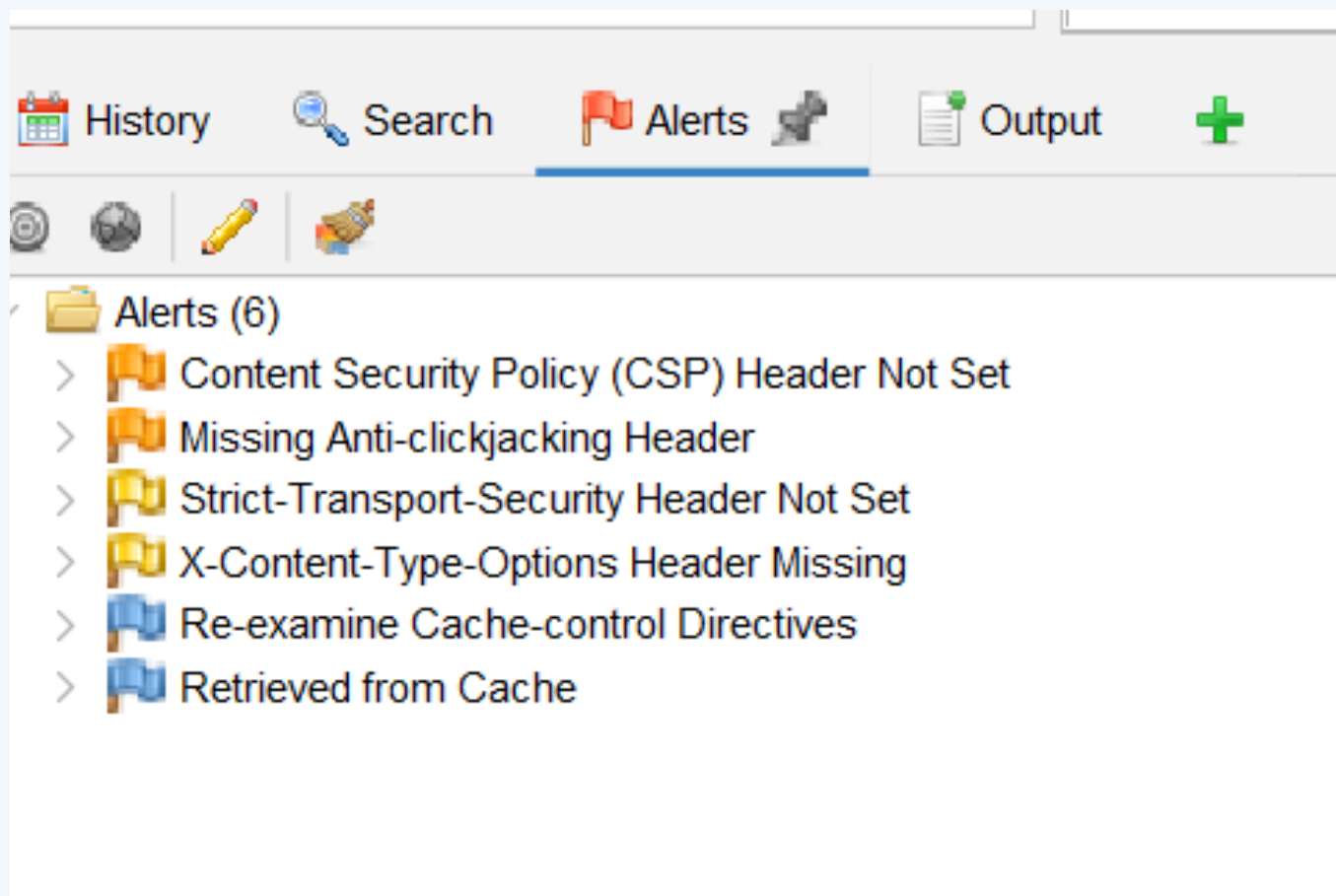


Figure 5: OWASP ZAP Alerts panel displaying the vulnerabilities identified during the passive security assessment.

Analysis

The passive scan identified a total of six findings, including four low-risk vulnerabilities and two informational observations. The detected issues mainly relate to missing HTTP security headers that improve browser security and help protect web applications against common attacks. No high-risk or critical vulnerabilities were identified during the assessment.

Vulnerability Finding 1

Title

Vulnerability Finding 1 – Content Security Policy (CSP) Header Not Set

OWASP ZAP detected that the Content Security Policy (CSP) header is missing. This header helps protect web applications against Cross-Site Scripting (XSS) and other code injection attacks.

Risk Level

Low

Configure the web server to include a Content-Security-Policy header to restrict the loading of untrusted resources and improve browser security.

Content Security Policy (CSP) Header Not Set
URL: <https://example.com/>
Risk:  Medium
Confidence: High
Parameter:
Attack:
Evidence:
CWE ID: 693
WASC ID: 15
Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)
Alert Reference: 10038-1
Input Vector:
Description:
Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — content types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
Other Info:
Solution:
Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header.
Reference:

8080

Current Status 

Figure 6: Content Security Policy (CSP) Header Not Set.

Additional Security Header Findings

1. Missing Anti-clickjacking Header

Description

OWASP ZAP identified that the website does not implement an anti-clickjacking header such as X-Frame-Options or the frame-ancestors directive in the Content Security Policy. This could allow the website to be embedded within another webpage.

Risk Level

Low

Recommendation

Configure the web server to send the X-Frame-Options header or implement the frame-ancestors directive within the Content Security Policy.

Missing Anti-clickjacking Header

URL:

https://example.com/

Risk:

Medium

Confidence:

Medium

Parameter:

x-frame-options

Attack:

Evidence:

CWE ID:

1021

WASC ID:

15

Source:

Passive (10020 - Anti-clickjacking Header)

Alert Reference:

10020-1

Input Vector:

Description:

The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.

Other Info:

Solution:

Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app. If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.

Figure 7: Missing Anti-clickjacking Header identified by OWASP ZAP

2. Strict-Transport-Security (HSTS) Header Not Set

Description

The HTTP Strict Transport Security (HSTS) header was not present in the server response. HSTS helps ensure browsers always communicate with the website over HTTPS.

Strict-Transport-Security Header Not Set

L:

https://example.com/

k:

Low

nfidence:

High

rameter:

ack:

ndence:

VE ID:

319

ASC ID:

15

urce:

Passive (10035 - Strict-Transport-Security Header)

rt Reference:

10035-1

ut Vector:

escription:

HTTP Strict Transport Security (HSTS) is a web security policy mechanism whereby a web server declares that complying user agents (such as a web browser) are to interact with it using only secure HTTPS connections (i.e. HTTP layered over TLS/SSL). HSTS is an IETF standards track protocol and is specified in RFC 6797.

ther Info:

olution:

Ensure that your web server, application server, load balancer, etc. is configured to enforce Strict-Transport-Security.

eference:

Figure 8: Missing Strict-Transport-Security (HSTS) header.

Other Info:

This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type. At "High" threshold this scan rule will not alert on client or server error responses.

Solution:

Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages. If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.

Reference:

[https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/cc622941\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/cc622941(v=vs.85))

Re-examine Cache-control Directives

URL: https://example.com/
 Risk: Informational
 Confidence: Low
 Parameter: cache-control
 Attack:
 Evidence:
 CWE ID: 525
 WASC ID: 13
 Source: Passive (10015 - Re-examine Cache-control Directives)
 Input Vector:

Description:

The cache-control header has not been set properly or is missing, allowing the browser and proxies to cache content. For static assets like css, js, or image files this might be intended, however, the resources should be reviewed to ensure that no sensitive content will be cached.

Other Info:

Solution:

For secure content, ensure the cache-control HTTP header is set with "no-cache, no-store, must-revalidate". If an asset should be cached consider setting the directives "public, max-age, immutable".

Reference:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html#web-content-caching

Retrieved from Cache

URL:

https://example.com/

Risk:

Informational

Confidence:

Medium

Parameter:

Attack:

Evidence:

Age: 7039

CWE ID:

525

WASC ID:

Source:

Passive (10050 - Retrieved from Cache)

Alert Reference:

10050-2

Input Vector:

Description:

The content was retrieved from a shared cache. If the response data is sensitive, personal or user-specific, this may result in sensitive information being leaked. In some cases, this may even result in a user gaining complete control of the session of another user, depending on the configuration of the caching components in use in their environment. This is primarily an issue where caching servers such as "proxy" caches are configured on the local network. This configuration is typically found in corporate or educational environments, for instance.

Other Info:

The presence of the 'Age' header indicates that a HTTP/1.1 compliant caching server is in use.

Solution:

Validate that the response does not contain sensitive, personal or user-specific information. If it does, consider the use of the following HTTP response headers, to limit, or prevent the content being stored and retrieved from the cache by another user:

Cache-Control: no-cache, no-store, must-revalidate, private

Reference:

8080

Current Status

Figure 11: Re-examine Cache-Control Directives.

Security Recommendations

Finding	Recommendation
Content Security Policy Header Not Set	Implement a Content-Security-Policy (CSP) header to reduce XSS risks.
Missing Anti-clickjacking Header	Configure the X-Frame-Options header or use the frame-ancestors CSP directive.
Strict-Transport-Security Header Not Set	Enable HSTS to enforce secure HTTPS connections.
X-Content-Type-Options Header Missing	Configure the X-Content-Type-Options: nosniff response header.
Cache-Related Findings	Review Cache-Control directives and apply appropriate caching policies for sensitive content.

Final Recommendation

Implementing the above recommendations will strengthen the security posture of the web application by improving browser-side protection, enforcing secure communication, and following OWASP security best practices. Regular vulnerability assessments should also be conducted to identify and address newly emerging security issues.

Conclusion

This project successfully demonstrated the process of performing a web application vulnerability assessment using OWASP ZAP. A demo website was developed, hosted locally, and assessed using passive scanning techniques. The assessment identified several low-risk security issues, primarily related to missing HTTP security headers and browser security best practices.

Although no critical or high-risk vulnerabilities were identified, the findings highlight the importance of implementing secure configurations to strengthen web application security. Appropriate recommendations have been provided to address each identified issue.

This internship task provided valuable hands-on experience with industry-standard security tools, including OWASP ZAP, Nmap, Mozilla Firefox, and Visual Studio Code. It also improved practical knowledge of vulnerability assessment, security analysis, and professional report documentation.

Overall, this project enhanced my understanding of web application security and demonstrated the importance of proactive security testing in identifying and mitigating potential risks.

References

- OWASP Foundation. OWASP Zed Attack Proxy (ZAP) Documentation.
- OWASP Foundation. OWASP Top 10 Web Application Security Risks.
- Nmap Project. Nmap Reference Guide.
- Mozilla Developer Network (MDN). HTTP Security Headers Documentation.
- Visual Studio Code Documentation.
- Mozilla Firefox Documentation.